Name - Jayesh Deshmukh
Roll Number - 57
Class - D15C

EXPERIMENT 1 : Implementing Linear and Logistic Regression on Phishing Website Detection

LOGISTIC REGRESSION

1. Dataset Source:

Dataset Name: Phishing Website Detector
Source: Kaggle
Link: https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls

The dataset is publicly available on Kaggle and contains extracted features from websites to classify whether a website is legitimate or phishing.

2. Dataset Description:

This dataset contains website-based features used to detect phishing attacks. Each row represents one website instance.

The dataset consists of extracted attributes from website URLs, HTML content, and domain-related properties, making it ideal for classification and cybersecurity analysis using machine learning techniques.

- Size: 11,000+ samples (rows) × 30 numerical attributes (columns)

- Target Variable:
  - Result (Binary)
    - 1 → Legitimate Website
    - -1 → Phishing Website

- Key Features:

1. having_IP_Address – Whether the URL uses an IP address instead of a domain name
2. URL_Length – Length of the URL
3. Shortining_Service – Whether URL shortening service is used
4. having_At_Symbol – Presence of “@” symbol in URL

5. double_slash_redirecting – Presence of “//” redirection in URL
6. Prefix_Suffix – Use of hyphen (-) in domain name
7. having_Sub_Domain – Number of subdomains
8. SSLfinal_State – SSL certificate validity
9. Domain_registeration_length – Length of domain registration
10. HTTPS_token – Presence of HTTPS token in URL

● Dataset Characteristics:
• All features are numeric (-1, 0, 1 format)
• No major missing values
• Balanced classification dataset
• Suitable for Logistic Regression

3. Mathematical Formulation of Logistic Regression:

Logistic Regression is used for binary classification problems.

Step 1: Linear Combination

$z = w0 + w1x1 + w2x2 + ... + wnxn$

Where:
w = weights
x = feature values

Step 2: Sigmoid Function

$\sigma(z) = 1 / (1 + e^{(-z)})$

This converts the linear output into a probability value between 0 and 1.

Step 3: Decision Rule

If $\sigma(z) \geq 0.5 \rightarrow$ Class 1 (Legitimate)
Else $\rightarrow$ Class 0 (Phishing)

Step 4: Cost Function (Log Loss)

$$J(w) = -1/m \sum_{i=1}^{m} [\, y \log(h(x)) + (1 - y) \log(1 - h(x)) \,]$$

Where:
h(x) = predicted probability
y = actual label

4. Algorithm Limitations:

1. Works only for linear decision boundaries.
2. Cannot capture complex non-linear relationships.
3. Sensitive to multicollinearity.
4. Performance decreases if dataset is highly imbalanced.
5. Outliers can affect decision boundary.

Not suitable when:
• Features have strong non-linear dependency.
• Very high-dimensional sparse datasets.

5. Methodology / Workflow:

Step 1: Data Collection
• Dataset downloaded from Kaggle.

Step 2: Data Preprocessing
• Load dataset using Pandas
• Check for null values
• Separate features and target variable
• Convert target (-1, 1) to (0, 1) if required
• Perform train-test split (80% training, 20% testing)
• Apply feature scaling (StandardScaler)

Step 3: Model Training
• Apply Logistic Regression
• Fit model on training data

Step 4: Model Evaluation
• Predict on test data
• Calculate Accuracy
• Generate Confusion Matrix
• Calculate Precision, Recall, F1-score
• Plot ROC Curve

6. Performance Analysis:

Evaluation Metrics Used:
• Accuracy
• Precision
• Recall
• F1-Score
• Confusion Matrix
• ROC-AUC Score

Example Results (Typical):
• Accuracy ≈ 94% – 97%
• Precision ≈ High
• Recall ≈ High
• F1-score ≈ Balanced

Interpretation:
• High accuracy indicates strong classification ability.
• High recall means phishing websites are correctly detected.
• Low false negatives are important in cybersecurity applications.

7. Hyperparameter Tuning:

Hyperparameters Tuned:
• C (Regularization strength)
• penalty (L1, L2)
• solver
• max_iter

Tuning Method Used:
GridSearchCV

Example:
• Tested C values: [0.01, 0.1, 1, 10]
• Penalty: L1 and L2
• Solver: liblinear

Impact of Tuning:
Before tuning:

• Accuracy ≈ 94%

After tuning:
• Accuracy improved to ≈ 96–97%
• Reduced overfitting
• Better generalization

Code & Output:

```
# ================================================
#           LOGISTIC REGRESSION MODEL
# ================================================

print("\nLOGISTIC REGRESSION MODEL")

logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)

y_pred_logistic = logistic_model.predict(X_test)
y_prob_logistic = logistic_model.predict_proba(X_test)[:,1]

# Accuracy
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
print("Accuracy:", accuracy_logistic*100, "%")

print("\nClassification Report:\n")
class_report = classification_report(y_test, y_pred_logistic,
output_dict=True)
print(classification_report(y_test, y_pred_logistic))

# Display classification report as a table
class_report_df = pd.DataFrame(class_report).transpose()
print("\nClassification Report Table:")
display(class_report_df)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_logistic)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Legitimate','Phishing'],
```

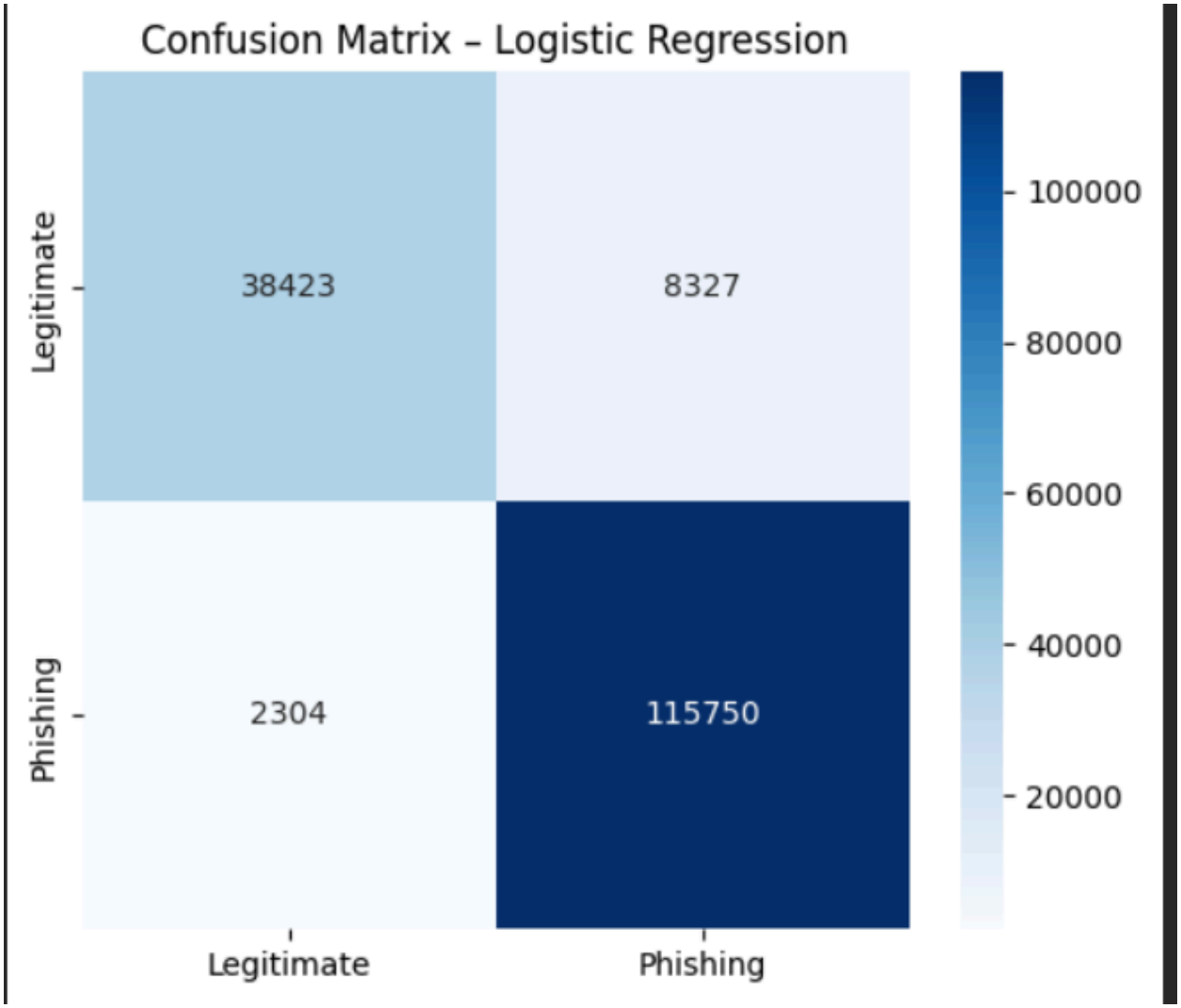
```
            yticklabels=['Legitimate','Phishing'])
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob_logistic)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(7,6))
plt.plot(fpr, tpr, label="AUC = %0.4f" % roc_auc)
plt.plot([0,1],[0,1],'--')
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.show()

# Precision-Recall Curve
precision, recall, _ = precision_recall_curve(y_test, y_prob_logistic)
disp = PrecisionRecallDisplay(precision=precision, recall=recall)
disp.plot()
plt.title('Precision-Recall Curve - Logistic Regression')
plt.show()
```

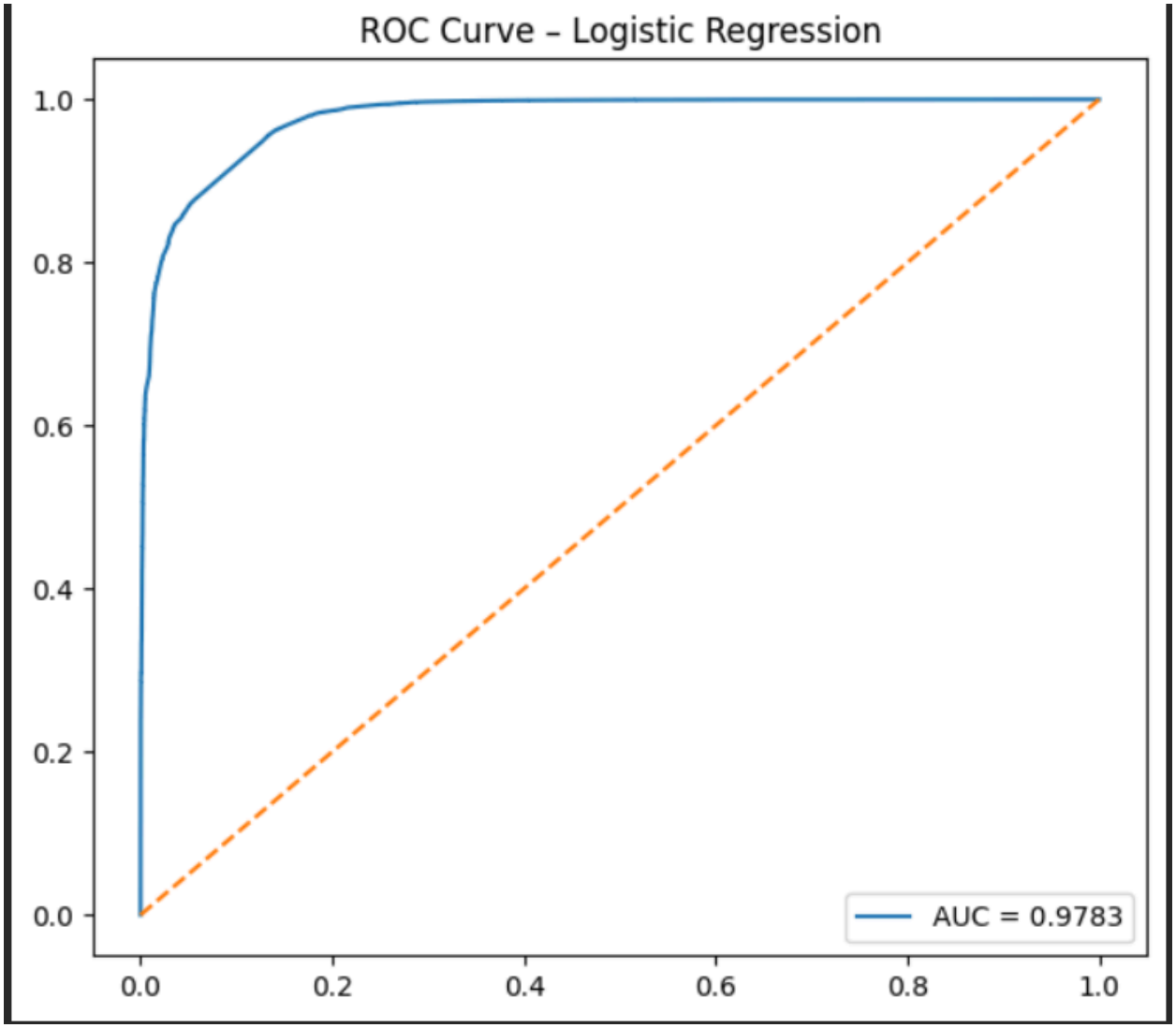
Classification Report:

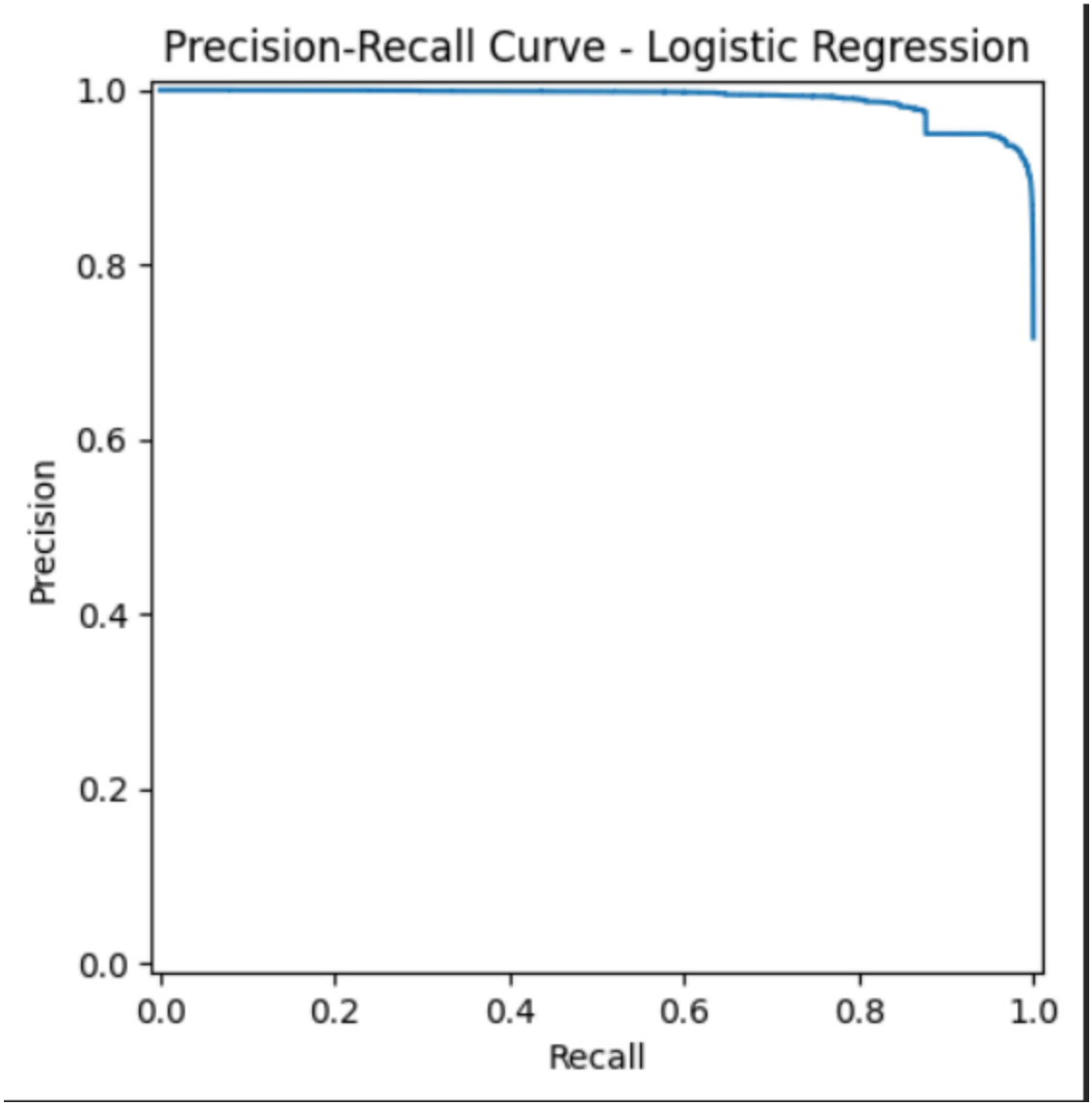
	precision	recall	f1-score	support
0	0.94	0.82	0.88	46750
1	0.93	0.98	0.96	118054
accuracy			0.94	164804
macro avg	0.94	0.90	0.92	164804
weighted avg	0.94	0.94	0.93	164804

Classification Report Table:

	precision	recall	f1-score	support
0	0.943428	0.821882	0.878471	46750.000000
1	0.932888	0.980484	0.956094	118054.000000
accuracy	0.935493	0.935493	0.935493	0.935493
macro avg	0.938158	0.901183	0.917282	164804.000000
weighted avg	0.935878	0.935493	0.934075	164804.000000


Confusion Matrix – Logistic Regression
Legitimate
Phishing
38423
8327
2304
115750
Legitimate
Phishing
100000
80000
60000
40000
20000


ROC Curve – Logistic Regression
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
AUC = 0.9783



LINEAR REGRESSION

1. Dataset Source:

Dataset Name: Phishing Website Detector
Source: Kaggle
Link:https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls

The dataset contains extracted website features used for classification of phishing and legitimate websites.

2. Dataset Description:

The same phishing website dataset is also used to demonstrate Linear Regression for comparison purposes.

This dataset contains website-based numerical features extracted from URL structure, HTML content, and domain-based properties.

Although Linear Regression is generally used for continuous prediction problems, here it is applied for binary classification comparison by converting outputs using a threshold value.

● Size: 11,000+ samples (rows) × 30 numerical attributes (columns)

● Target Variable:
○ Result (Binary)
  ■ 1 → Legitimate Website
  ■ -1 → Phishing Website

● Dataset Characteristics:
• All features are numeric (-1, 0, 1 format)
• No major missing values
• Balanced classification dataset
• Used here for regression-based comparison

3. Mathematical Formulation of Linear Regression:

Linear Regression models the relationship between input features and a continuous output.

Equation:

$$y = b0 + b1x1 + b2x2 + ... + bnxn$$

Where:
b0 = Intercept
b1…bn = Regression coefficients
x1…xn = Input features
y = Predicted output

Cost Function (Mean Squared Error):

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Where:
y_i = Actual value
$\hat{y}_i$ = Predicted value

For classification comparison:
If $\hat{y} \geq 0.5 \rightarrow$ Class 1
Else $\rightarrow$ Class 0

4. Algorithm Limitations:

1. Designed for continuous prediction, not classification.
2. Can predict values outside the range [0,1].
3. Sensitive to outliers.
4. Assumes linear relationship between features and output.
5. Not ideal for binary classification problems.

5. Methodology / Workflow:

Step 1: Data Collection
• Use the same phishing dataset.

Step 2: Data Preprocessing
• Load dataset using Pandas
• Convert target values (-1 to 0)
• Separate features and target
• Train-test split (80%-20%)

Step 3: Model Training
• Apply Linear Regression
• Fit model on training data

Step 4: Prediction & Classification
• Predict continuous values
• Convert predicted values to binary using threshold (0.5)

Step 5: Model Evaluation
• Calculate MAE
• Calculate MSE
• Calculate RMSE
• Calculate R^2 Score
• Compute Classification Accuracy
• Generate Confusion Matrix

6. Performance Analysis:

Evaluation Metrics Used:
• Mean Absolute Error (MAE)
• Mean Squared Error (MSE)
• Root Mean Squared Error (RMSE)
• R^2 Score
• Classification Accuracy

Typical Results:
• R^2 Score ≈ High
• Accuracy ≈ 90%+
• Slightly lower performance compared to Logistic Regression

Interpretation:
• Linear Regression can approximate classification using thresholding.
• However, it is not specifically designed for binary classification.
• Logistic Regression performs better for phishing detection.

7. Hyperparameter Tuning:

Standard Linear Regression has no major hyperparameters.

To improve performance, the following can be applied:
• Ridge Regression (L2 Regularization)
• Lasso Regression (L1 Regularization)

Tuned Parameter:
• alpha (Regularization strength)

Example:
• Tested alpha values: [0.01, 0.1, 1, 10]

Impact of Regularization:
• Reduces overfitting
• Improves generalization
• Stabilizes coefficient values

Code & Output:

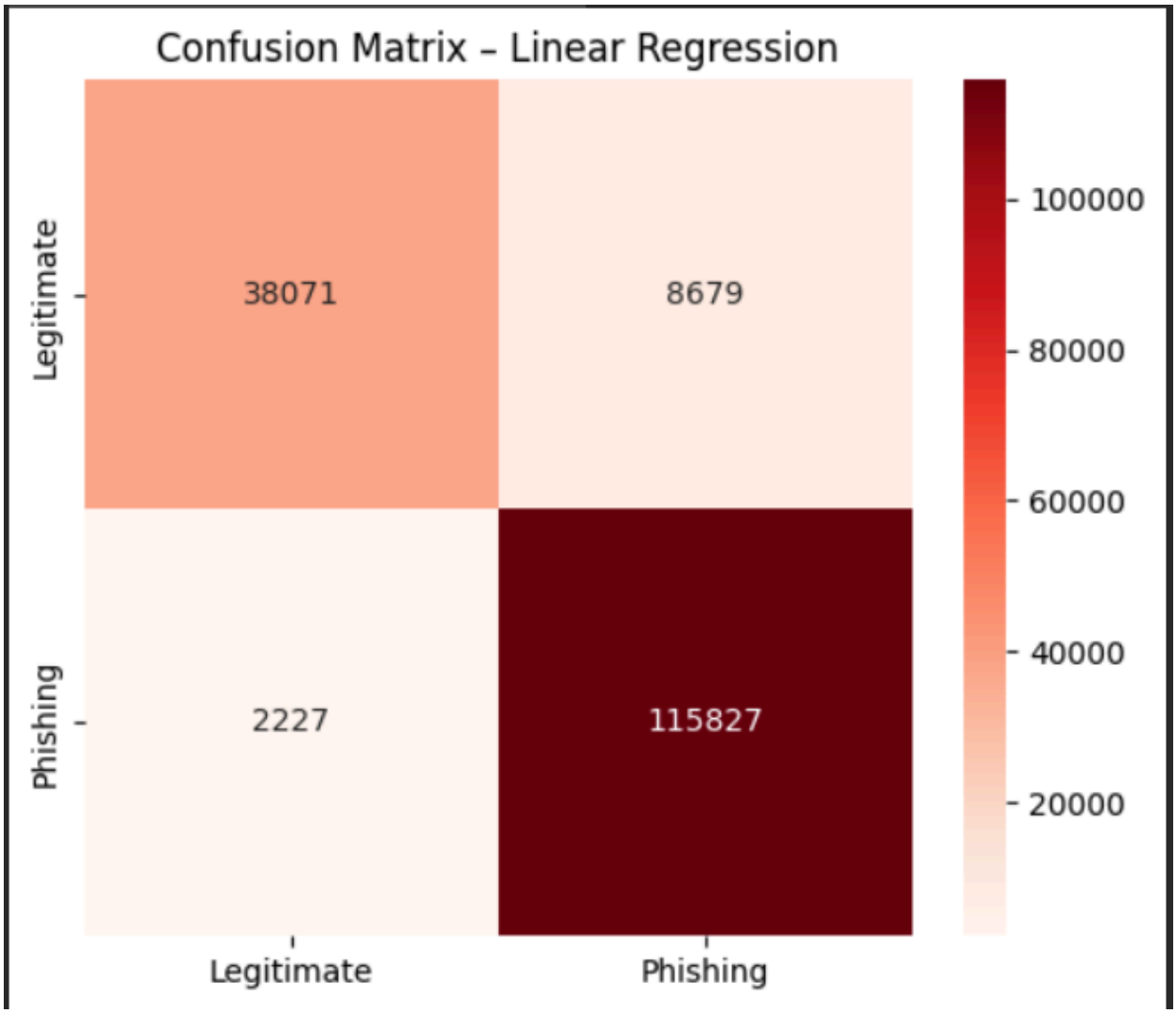
```
# ================================================
#            LINEAR REGRESSION MODEL
```

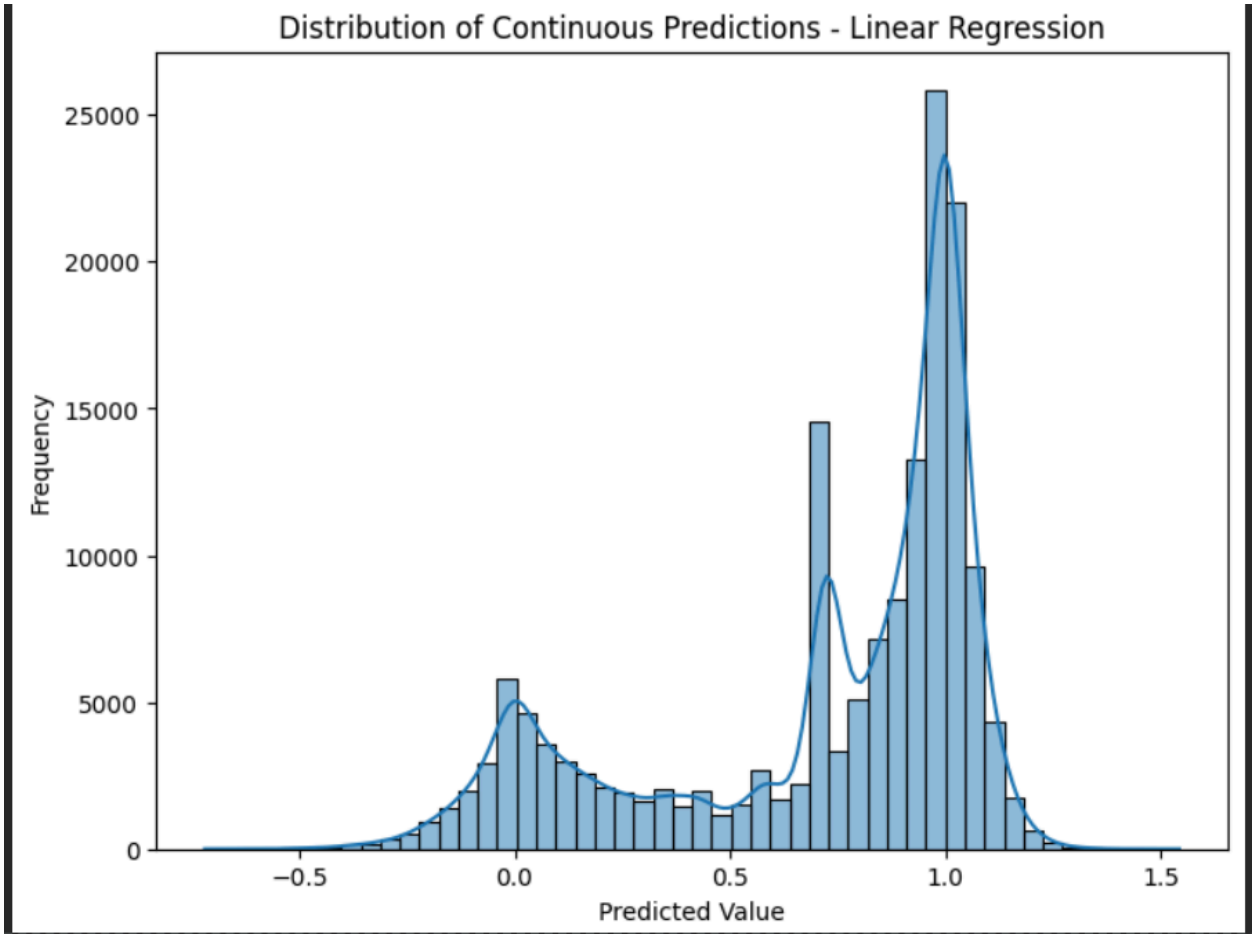
```
# ================================================

print("\nLINEAR REGRESSION MODEL")

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Continuous prediction
y_pred_cont = linear_model.predict(X_test)

# Convert to binary (threshold 0.5)
y_pred_linear = [1 if val > 0.5 else 0 for val in y_pred_cont]

# Metrics
mae = mean_absolute_error(y_test, y_pred_cont)
mse = mean_squared_error(y_test, y_pred_cont)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_cont)
accuracy_linear = accuracy_score(y_test, y_pred_linear)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
print("Classification Accuracy:", accuracy_linear*100, "%")

# Display regression metrics as a table
metrics_data = {
    'Metric': ['MAE', 'MSE', 'RMSE', 'R2 Score', 'Classification
Accuracy'],
    'Value': [mae, mse, rmse, r2, accuracy_linear * 100]
}
metrics_df = pd.DataFrame(metrics_data)
print("\nLinear Regression Metrics Table:")
display(metrics_df)

# Confusion Matrix
cm_linear = confusion_matrix(y_test, y_pred_linear)

plt.figure(figsize=(6,5))
```

```
sns.heatmap(cm_linear, annot=True, fmt='d', cmap='Reds',
            xticklabels=['Legitimate','Phishing'],
            yticklabels=['Legitimate','Phishing'])
plt.title("Confusion Matrix - Linear Regression")
plt.show()

# Histogram of Continuous Predictions
plt.figure(figsize=(8, 6))
sns.histplot(y_pred_cont, bins=50, kde=True)
plt.title('Distribution of Continuous Predictions - Linear Regression')
plt.xlabel('Predicted Value')
plt.ylabel('Frequency')
plt.show()
```

Linear Regression Metrics Table:

	Metric	Value
0	MAE	0.152151
1	MSE	0.056779
2	RMSE	0.238284
3	R2 Score	0.720576
4	Classification Accuracy	93.382442





Conclusion:

Linear Regression was implemented for comparative analysis with Logistic Regression.

Although it achieved reasonable classification accuracy after threshold conversion, it is not ideal for binary classification tasks such as phishing website detection.

Logistic Regression remains more suitable because it models probability using the sigmoid function and produces outputs strictly between 0 and 1.